Implementation of Hidden Markov Model (HMM) and Viterbi Algorithm

JOU-YI LEE
UC Santa Cruz, M.S. in NLP
65 Rio Robles E
San Jose, CA 95134
jlee1039@ucsc.edu

Abstract

This assignment focuses on building a Part-of-Speech (POS) tagger for English using a Hidden Markov Model (HMM). The task involves deriving the Viterbi algorithm for sequence tagging, implementing a bigram HMM with add-α smoothing, and performing Viterbi decoding to predict the most likely sequence of POS tags for given sentences. The dataset used is the Wall Street Journal (WSJ) portion of the Penn Treebank, divided into training, development, and test sets. The POS tagger is evaluated based on tagging accuracy, precision, recall, and F1 scores. A baseline model, which assigns the most frequent tag to each word, is implemented for comparison. The project explores optimization strategies for hyperparameters and discusses potential improvements in tagging performance and computational efficiency. The results highlight the effectiveness of HMMs in supervised POS tagging and identify challenges in disambiguating similar tags.

1 Deriving the Viterbi Algorithm

1. Explain why $\arg\max_t P(t \mid w) = \arg\max_t \log(P(t, w))$.
2. Derive the recurrence formula for $\pi_j(t_j)$ and show how it can be computed using π_{j-1}.
3. Provide an algorithm for computing $\hat{t}$ and analyze its time complexity.
4. Discuss the semiring formulation of the Viterbi algorithm and explain how it generalizes the Forward algorithm.

1.1 Why does $\arg\max_t P(t \mid w) = \arg\max_t \log(P(t, w))$?

Maximizing the conditional probability $P(t \mid w)$ is our goal. According to Bayes' theorem, we have:

$$P(t \mid w) = \frac{P(t, w)}{P(w)}.$$

Since $P(w)$ is independent of the tag sequence t, maximizing $P(t \mid w)$ is equivalent to maximizing the joint probability $P(t, w)$. Furthermore, the $\log$ function is monotonic, meaning it does not change the location of the maximum.

Thus, maximizing $P(t, w)$ and maximizing $\log(P(t, w))$ are equivalent. Formally, we can write:

$$\arg\max_t P(t \mid w) = \arg\max_t \log(P(t, w)).$$

2. Proof that $\pi_j(t_j)$ can be computed using π_{j-1}

Definition:

- $\pi_j(t_j)$ represents the log probability of the highest-scoring tag sequence of length j that ends in tag t_j:

$$\pi_j(t_j) = \max_{t_1,\ldots,t_{j-1}} \sum_{i=1}^{j} \text{score}(w, i, t_i, t_{i-1})$$

- The score function is defined as:

$$\text{score}(w, i, t_i, t_{i-1}) = \log(P(w_i \mid t_i)) + \log(P(t_i \mid t_{i-1})).$$

Derivation: Consider the recurrence relation by separating the j-th tag t_j from its preceding tag t_{j-1}:

$$\pi_j(t_j) = \max_{t_1,\ldots,t_{j-1}} \Big(\sum_{i=1}^{j-1} \text{score}(w, i, t_i, t_{i-1}) + \text{score}(w, j, t_j, t_{j-1}) \Big). \quad (1)$$

Observe that:

$$\sum_{i=1}^{j-1} \text{score}(w, i, t_i, t_{i-1}) = \pi_{j-1}(t_{j-1}).$$

Substituting this into the equation:

$$\pi_j(t_j) = \max_{t_{j-1}} \Big(\pi_{j-1}(t_{j-1}) + \text{score}(w, j, t_j, t_{j-1}) \Big).$$

Result: The final recurrence formula is:

$$\pi_j(t_j) = \max_{t_{j-1}} \Big(\pi_{j-1}(t_{j-1}) + \log(P(w_j \mid t_j)) + \log(P(t_j \mid t_{j-1}))\Big). \quad (2)$$

Significance:

- $\pi_j(t_j)$ can be recursively computed from π_{j-1}.
- This recurrence formula forms the core of the Viterbi algorithm, enabling dynamic programming to find the optimal path efficiently.

3. Algorithm for Computing $\hat{t}$ and Time Complexity Analysis

Problem Description: We aim to compute the optimal tag sequence $\hat{t}$ that maximizes the following equation:

$$\hat{t} = \arg\max_{t_1,\ldots,t_n} \sum_{i=1}^{n} \text{score}(w, i, t_i, t_{i-1})$$

where:

$$\pi_j(t_j) = \max_{t_{j-1}} \Big(\pi_{j-1}(t_{j-1}) + \log(P(w_j \mid t_j)) + \log(P(t_j \mid t_{j-1}))\Big). \quad (3)$$

Algorithm:

1. **Input:**
   - Word sequence $w = [w_1, w_2, \ldots, w_n]$
   - Set of possible tags $T = \{t_1, t_2, \ldots, t_k\}$,
   - Emission probabilities $P(w_i \mid t_i)$
   - Transition probabilities $P(t_i \mid t_{i-1})$
2. **Initialization:** Define a dynamic programming table $V[j][t]$, where $V[j][t]$ stores the log probability of the highest-scoring tag sequence ending in tag t at position j. Initialize:

$$V[1][t] = \log(P(w_1 \mid t)) + \log(P(t \mid \text{START})). \quad (4)$$

3. **Recursion:** For each word w_j (from the second to the n-th word) and each possible tag $t \in T$, compute:

$$V[j][t] = \max_{t' \in T} \Big(V[j-1][t'] + \log(P(t \mid t')) + \log(P(w_j \mid t))\Big). \quad (5)$$

   Store the predecessor tag t' that maximizes this value in a backpointer table.
4. **Termination:** Find the highest log probability among all possible final tags:

$$\hat{t}_n = \arg\max_{t \in T} V[n][t].$$

5. **Backtracking:** Use the backpointer table to trace back the optimal sequence $\hat{t}$ from $\hat{t}_n$ to $\hat{t}_1$.

Time Complexity Analysis:

- **Initialization:** The cost is $O(k)$, where k is the number of possible tags.
- **Recursion:** For each word w_j, and for each current tag t, we compute the maximum over k previous tags. This results in $O(k^2)$ operations per word. For n words, the total cost is $O(n \cdot k^2)$.
- **Termination and Backtracking:** Termination takes $O(k)$, and backtracking takes $O(n)$.

Overall Time Complexity: The dominant term comes from the recursion, making the overall time complexity $O(n \cdot k^2)$, where n is the length of the sequence, and k is the number of tags.

Conclusion: The algorithm efficiently computes the optimal tag sequence using dynamic programming, leveraging the recurrence relation in the Viterbi algorithm. The time complexity $O(n \cdot k^2)$ is manageable for moderate-sized tag sets and sentence lengths.

Relationship Between the Forward Algorithm and the Viterbi Algorithm

1. **Forward Algorithm:**
   - Used to compute the total probability of a sequence of states in a Hidden Markov Model (HMM).
   - Operations are primarily based on addition (summation) and multiplication.
2. **Viterbi Algorithm:**
   - Used to find the most likely sequence of states in an HMM.
   - Can be viewed as a specialized version of the Forward Algorithm based on a **semiring**.
   - In the Viterbi algorithm, "addition" is replaced by $\max$, and "multiplication" is replaced by addition ($+$).

Definition of a Semiring

A semiring $S = \langle A, \oplus, \otimes, 0_s, 1_s \rangle$ consists of:

- A: A set of elements.
- $\oplus$: The addition operation (in the Viterbi algorithm, this corresponds to $\max$).
- $\otimes$: The multiplication operation (in the Viterbi algorithm, this corresponds to $+$).
- 0_s: The additive identity (e.g., $-\infty$ in the Viterbi semiring).
- 1_s: The multiplicative identity (e.g., 0 in the Viterbi semiring).

In the Viterbi semiring:

$$\oplus = \max, \quad \otimes = +, \quad 0_s = -\infty, \quad 1_s = 0.$$

Explanation of the Formula

Goal Formula: The goal of the Viterbi algorithm is to find the most likely sequence of tags:

$$\hat{t} = \bigoplus_{t_0,\ldots,t_{n+1}} \bigotimes_{i=1}^{n} \text{score}(w, i, t_i, t_{i-1}),$$

Recurrence Formula: For each tag t_j, the best score can be recursively computed as:

$$\pi_j(t_j) = \bigoplus_{t_{j-1}} \Big(\pi_{j-1}(t_{j-1}) \otimes \text{score}(w, j, t_j, t_{j-1}) \Big).$$

Proof of the Recurrence Relation

$$\pi_j(t_j) = \bigoplus_{t_{j-1}} \Big(\pi_{j-1}(t_{j-1}) \otimes \text{score}(w, j, t_j, t_{j-1}) \Big).$$

Derivation:

1. $\pi_j(t_j)$ represents the best score for a sequence ending with tag t_j:

$$\pi_j(t_j) = \bigoplus_{t_1,\ldots,t_{j-1}} \bigotimes_{i=1}^{j} \text{score}(w, i, t_i, t_{i-1}).$$

2. Separate the j-th tag t_j from the preceding tags:

$$\pi_j(t_j) = \bigoplus_{t_{j-1}} \Big(\pi_{j-1}(t_{j-1}) \otimes \text{score}(w, j, t_j, t_{j-1}) \Big).$$

3. In this formulation:
   - $\pi_{j-1}(t_{j-1})$: The best score for the sequence up to t_{j-1}.
   - $\text{score}(w, j, t_j, t_{j-1})$: The transition and emission score for moving from t_{j-1} to t_j.

Properties of the Semiring

The derivation uses the following properties of the semiring:

1. **Associativity:**

$$(a \otimes b) \otimes c = a \otimes (b \otimes c),$$

2. **Commutativity of $\oplus$:**

$$a \oplus b = b \oplus a,$$

3. **Distributivity:**

$$a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c),$$

4. **Identity Elements:**

$$a \oplus 0_s = a, \quad a \otimes 1_s = a,$$

Algorithm Modifications

To solve Eq. (2), the following modifications are needed:

- **Addition Operation ($\oplus$):** Replace with $\max$.
- **Multiplication Operation ($\otimes$):** Replace with $+$.
- **Initialization:** Set all initial values to the additive identity $0_s = -\infty$.
- **Termination:** Use the final step to compute the maximum ($\max$) to find the best sequence.

2 Data Preprocessing

I used the `load_treebank_splits` function to split the dataset into Train (Folders 00–18), Dev (Folders 19–21), and Test (Folders 22–24). The data is read from the specified directory and converted into a formatted list of (word, tag) tuples.

3 HMM Model Construction

I defined the `build_hmm` function to construct the HMM model. The HMM computes the following:

- **Transition Counts (`transition_counts`)**: Counts the transitions between tags.
- **Emission Counts (`emission_counts`)**: Counts the occurrences of words given a tag.
- **State Counts (`state_counts`)**: Tracks the count of each tag state.

The model supports the complete tagging sequence from `<START>` to `<STOP>`.

4 Probability Calculation

The `calculate_probabilities` function is used to compute:

- **Transition Probabilities (`transition_probs`)**
- **Emission Probabilities (`emission_probs`)**

Additive smoothing ($\alpha = 1$) is applied to handle rare events. The <UNK> (unknown word) concept is introduced to avoid zero probabilities for unseen words. Using Maximum a Posteriori (MAP) estimation, the transition and emission probabilities are calculated.

5 Viterbi Algorithm

The Viterbi algorithm is a dynamic programming method used to find the most likely sequence of states in a Hidden Markov Model (HMM). It is commonly applied to tasks like POS tagging and speech recognition. Below is a detailed explanation of the `viterbi` function.

5.1 Initialization

For the first word in the sentence, the probability of each possible state (tag) is calculated as:

$$\text{dp}[0][\text{state}] = P(\text{state} \mid \texttt{<START>}) \cdot P(\text{word}_0 \mid \text{state})$$

Where $\text{dp}[t][\text{state}]$ represents the maximum probability of the sequence up to position t with the state being `state`.

5.2 Recursion

Starting from the second word, for each state s, consider all possible previous states s' and compute:

$$\text{dp}[t][\text{state}] = \max_{\text{prev_state}} (\text{dp}[t-1][\text{prev_state}] \cdot P(\text{state}$$

The `backpointer[t][state]` records the previous state that yields the highest probability, which is useful for tracing back the best path.

5.3 Termination

At the end of the sequence, the best final state is selected based on:

$$\text{dp}[n-1][\text{state}] \cdot P(\texttt{<STOP>} \mid \text{state})$$

where n is the length of the sentence.

5.4 Backtracking

Starting from the final state, use the `backpointer` to trace back the optimal sequence of states.

6 Performance Evaluation

The `evaluate` function compares the predicted tags against the true tags:

- **Metrics**: Accuracy, precision, recall, and F1 score are calculated using `sklearn.metrics.classification_report`.
- **Output**: Detailed classification metrics and confusion matrix are provided for each tag.

7 Results and Analysis

7.1 Performance Metrics

Tag	Precision	Recall	F1-Score	Support
#	0.25	1.00	0.40	22
$	0.87	0.99	0.92	1138
”	0.84	0.97	0.90	1380
(	0.86	0.85	0.86	239
)	0.72	0.90	0.80	240
,	1.00	0.97	0.98	9041
.	1.00	1.00	1.00	7035
:	0.97	0.83	0.89	959
CC	1.00	0.85	0.92	4287
CD	0.99	0.79	0.88	6000
DT	0.99	0.94	0.97	14937
EX	0.35	0.98	0.51	174
FW	0.00	0.70	0.00	37
IN	0.97	0.86	0.91	18138
...	(Additional tags are truncated for brevity)			

Table 1: Performance metrics for each tag

Tag	Description	Example	Tag	Description	Example	Tag	Description	Example
CC	coordinating conjunction	and, but , or	PDT	predeterminer	all, both	VBP	VBP verb non -3sg present	eat
CD	cardinal number	one, two	POS	possessive ending	's	VBZ	verb 3sg pres	eats
DT	determiner	a, the	PRP	personal pronoun	I, you , he	WDT	wh-determ.	which , that
EX	existential 'there'	there	PRP$	possess. pronoun	your, one's	WP	wh-pronoun	what , who
FW	foreign word	mea culpa	RB	adverb	quickly	WP$	wh-possess.	whose
IN	preposition/ subordin-conj	of, in, by	RBR	comparative adverb	faster	WRB	wh-adverb	how, where
JJ	adjective	yellow	RBS	superlatv. adverb	fastest	$	dollar sign	$
JJR	comparative adj	bigger	RP	particle	up, off	#	pound sign	#
JJS	superlative adj	wildest	SYM	symbol	+,%, &	“	left quote	‘ or “
LS	list item marker	1, 2, One	TO	“to”	to	”	right quote	’ or ”
MD	modal	can, should	UH	interjection	ah, oops	(	left paren	[, (, {, ⟨
NN	sing or mass noun	llama	VB	verb base form	eat	)	right paren	],), }, ⟩
NNS	noun, plural	llamas	VBD	verb past tense	ate	,	comma	,
NNP	proper noun, sing.	IBM	VBG	verb gerund	eating	.	sent-end punc	. ! ?
NNPS	proper noun, plu.	Carolinas	VBN	verb past part.	eaten	:	sent-mid punc	: ; ... – -

Figure 1: Penn Treebank POS Tagset

7.2 Key Metrics

- **Accuracy:** The overall accuracy of the POS tagger is **80%**, indicating that 4 out of 5 tags were correctly predicted.
- **Macro Average:** Precision: 0.51, Recall: 0.58, F1-Score: 0.51. This shows the average performance across all tags weighted equally.
- **Weighted Average:** Precision: 0.95, Recall: 0.80, F1-Score: 0.86. This highlights the model's performance weighted by the frequency of each tag, emphasizing common tags like `NN`, `DT`, and `IN`.

7.3 Insights by Tag

Frequent Tags Perform Well:

- Common punctuation marks (`.`, `,`): Achieve perfect or near-perfect precision and recall due to their straightforward nature.
- Specific POS Tags (`$`, `TO`, `DT`): High scores, reflecting consistent usage in language.
- Pronouns (`PRP`, `PRP$`): Precision and recall are high, indicating that the model effectively identifies pronouns and possessive pronouns.
- Nouns (`NN`): Precision: 0.97, Recall: 0.76. The model confidently predicts this tag.

Medium Performance Tags:

- Adjectives and Comparatives (`JJ`, `JJR`, `JJS`): Lower recall indicates difficulties in identifying less common adjectives or comparative forms.
- Verbs (`VB`, `VBD`, `VBZ`): Achieve balanced precision and recall, showing the model captures their nuances well.
- Prepositions (`IN`): High performance due to their frequency and distinctiveness in sentences.

Low Performance Tags:

- Unknown or Compound Tags (`IN|RB`, `JJ|NN`, `NN|POS`): Precision and recall are 0 due to limited or absent training data for these tags.
- Special Characters (`#`, `LS`): High recall (e.g., 1.00 for #) but low precision indicates over-prediction.

8 Conclusion

The implemented HMM model, combined with the Viterbi algorithm, achieves efficient and accurate POS tagging. Additive smoothing ($\alpha = 1$) and the introduction of the `<UNK>` token provide robustness against rare and unseen words. Frequent tags like `NN`, `DT`, and `IN` perform well due to their abundance in the training data, while rare and ambiguous tags remain challenging. Future improvements could focus on:

- Increasing training data for rare tags.
- Refining feature engineering to handle ambiguous tags.
- Optimizing smoothing parameters for unknown or infrequent cases.